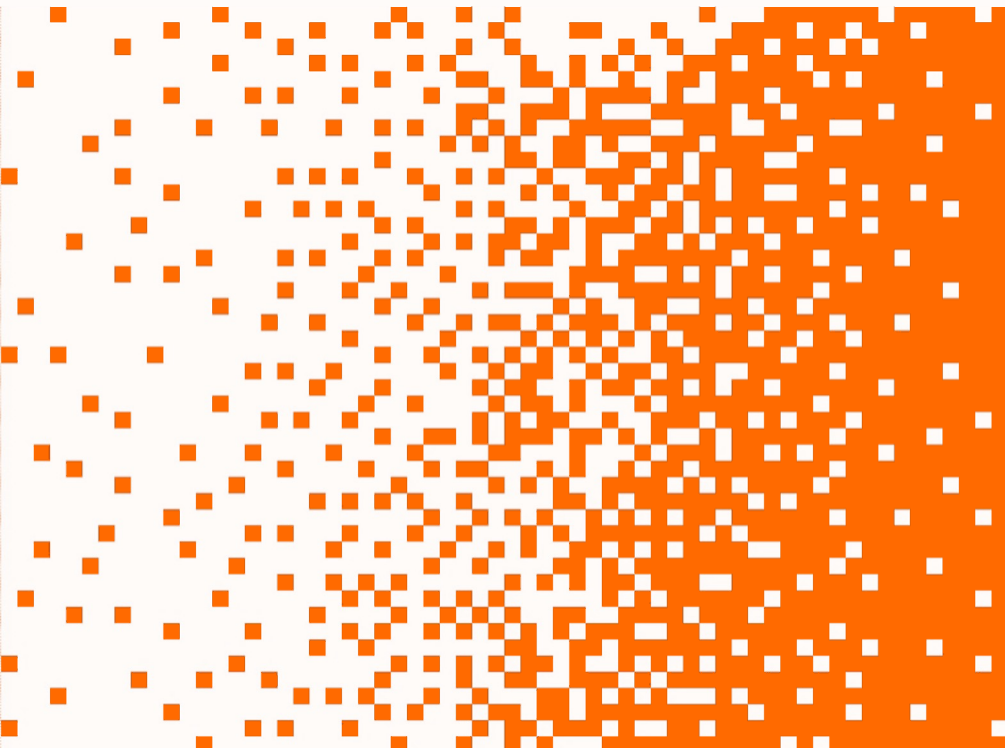


Datenbankentwurf am Fallbeispiel VeloCity

Vom Fachkonzept zur abgesicherten Datenhaltung — Anforderungen, ERM, Normalisierung, Relationenmodell, DDL, Zugriffsschutz



Was Sie nach dieser Einheit können

Der Entwurfszyklus ist kein Ritual, sondern eine Reihenfolge mit Grund: jeder Schritt trifft Entscheidungen, die der nächste voraussetzt. Wer Spalten entwirft, bevor die Geschäftsregeln stehen, baut ein Modell für eine Wirklichkeit, die er nicht geprüft hat.

Modellieren

- Aus einem Fachtext ein ERM ableiten
- Kardinalitäten als prüfbare Aussagen formulieren
- Surrogat- und Fachschlüssel unterscheiden

Prüfen

- Funktionale Abhängigkeiten benennen
- 1NF bis 3NF am Beispiel durchführen
- Erkennen, wo Normalisierung nicht genügt

Umsetzen

- Geschäftsregeln in Constraints übersetzen
- Zugriffsschutz in der Datenbank verankern
- Die Umsetzung nachweisen statt behaupten

Fallstudie VeloCity, Schema velocity auf supabase.butscher.cloud. Quellen im Repository unter doku/datenmodell/.

Der Entwurfszyklus gibt die Gliederung vor

Die Reihenfolge ist zwingend. Anforderungen vor Modell, Modell vor Relationen, Relationen vor DDL. Wer sie umdreht, bindet sich an eine Umsetzung, bevor er weiss, was umzusetzen ist.



Fallstudie VeloCity, Schema velocity auf supabase.butscher.cloud. Quellen im Repository unter doku/datenmodell/.

1 · Die Fallstudie

VeloCity vermietet Räder minutengenau

Ein Bike-Sharing-System in Würzburg. Kunden finden freie Räder auf einer Karte, entleihen sie, fahren und stellen sie wieder ab. Vielfahrer schliessen einen Tarif ab, der Freiminuten und Rabatt mitbringt.

Drei Fahrzeugklassen

City-Bike, Stadtrad ohne Motor
E-Bike Sport, Pedelec bis 25 km/h
E-Cargo Loader, Lastenrad bis 100 kg

Zwei Abstellarten

An einer festen Station, kostenfrei
Frei im Stadtgebiet gegen Zuschlag
Beides muss das Modell abbilden

Vier Tarife

Basis ohne Freiminuten
Student und OEPNV mit Kontingent
Premium mit Rabatt und Monatspreis

Fallstudie VeloCity, Schema velocity auf supabase.butscher.cloud. Quellen im Repository unter doku/datenmodell/.

Zehn Geschäftsregeln, die das Modell tragen müssen

Diese Regeln sind die fachliche Substanz. Entscheidend ist nicht, sie zu notieren, sondern dass jede einzelne am Ende erzwungen wird — von der Datenbank, nicht von der Anwendung.

1 GR1 Ein Rad ist höchstens einmal gleichzeitig ausgeliehen

partieller UNIQUE-Index

2 GR2 Ein Kunde hat höchstens vier aktive Ausleihen

Prüfung in der Funktion

3 GR3 Ein Kunde hat zu einem Zeitpunkt einen Tarif

EXCLUDE-Constraint

4 GR4 Preise und Konditionen überlappen sich nie

EXCLUDE-Constraint

5 GR5 Bepreist wird mit dem Preis zur Startzeit

in fn_ausleihe_beenden

Fallstudie VeloCity, Schema velocity auf supabase.butscher.cloud. Quellen im Repository unter doku/datenmodell/.

Die übrigen fünf Regeln und wo sie greifen

Nicht jede Regel lässt sich als Constraint formulieren. Drei brauchen Kontext, den eine Tabelle nicht hat: den angemeldeten Nutzer, das heutige Datum, den Zustand anderer Zeilen.

1	GR6 Angefangene Minuten werden aufgerundet	GENERATED-Spalte
2	GR7 Verbrauch übersteigt nie das Kontingent	CHECK-Constraint
3	GR8 Mindestalter 16 Jahre	in der API-Funktion
4	GR9 Nur der Kunde beendet seine Ausleihe	auth.uid()-Prüfung
5	GR10 Eine Rechnung je Kunde und Monat	UNIQUE-Constraint

Fallstudie VeloCity, Schema velocity auf supabase.butscher.cloud. Quellen im Repository unter doku/datenmodell/.

2 · Konzeptioneller Entwurf

Ein ERM ist eine Behauptung über die Wirklichkeit

Jede Kardinalität sagt etwas über das Geschäft aus und kann falsch sein. „Ein Kunde hat höchstens einen gültigen Tarif“ ist keine technische Festlegung, sondern eine Entscheidung des Fachbereichs.

Entität

Ein Ding, über das Daten gehalten werden
Im Text meist ein Substantiv
Beispiel: Kunde, Station, Ausleihe

Beziehung

Eine Verbindung zwischen Entitäten
Im Text meist ein Verb
Beispiel: Kunde tätigt Ausleihe

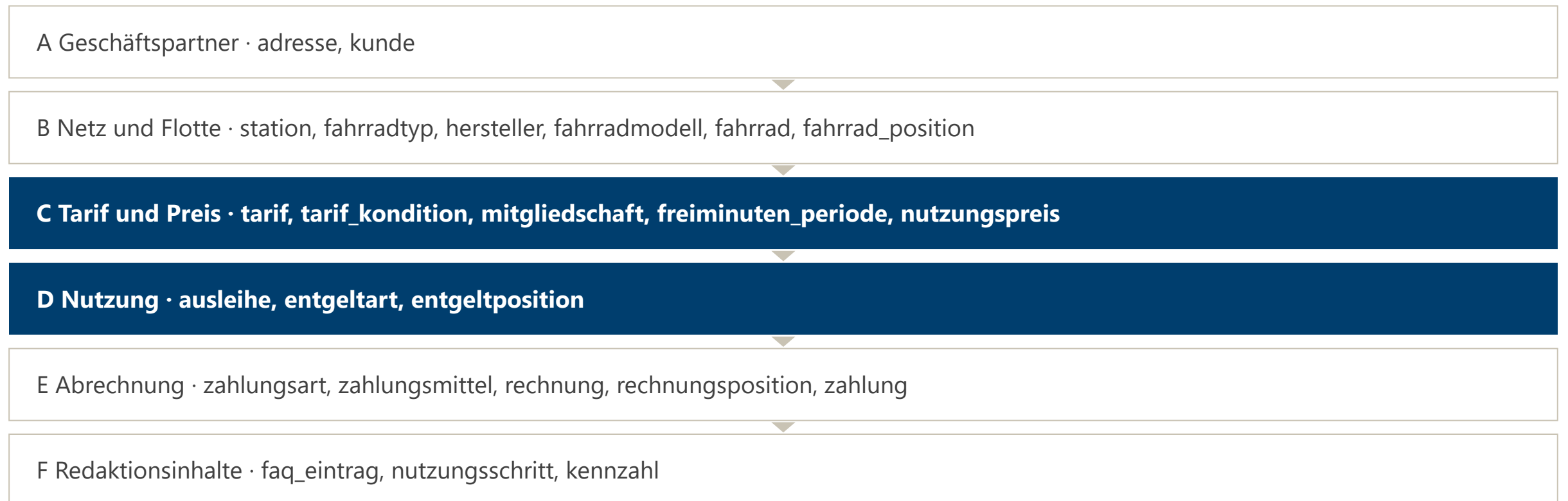
Kardinalität

Wie viele auf jeder Seite stehen
Die eigentliche fachliche Aussage
Muss mit dem Fachbereich geklärt sein

Fallstudie VeloCity, Schema velocity auf supabase.butscher.cloud. Quellen im Repository unter doku/datenmodell/.

Zehn Fachbereiche, davon sechs jetzt umgesetzt

Die Aufteilung folgt dem Geschäft, nicht der Technik. Bereiche, die zusammen geändert werden, gehören zusammen.



Fallstudie VeloCity, Schema velocity auf supabase.butscher.cloud. Quellen im Repository unter doku/datenmodell/.

Vier Entscheidungen, die den Unterschied machen

Ein Modell besteht nicht aus Tabellen, sondern aus Entscheidungen. Diese vier sind die, die man begründen können muss.

1 Adresse eigenständig wird von Kunde, Station, Lieferant und Lager gebraucht

keine vierfache Redundanz

2 Preise nicht am Typ ein Preis gilt für einen Zeitraum, nicht für immer

nutzungspreis mit daterange

3 Position als Satellit Stammdaten ruhig halten, Bewegungsdaten trennen

fahrrad_position 1:1

4 Freiminuten je Periode ein Zähler verliert seine Geschichte

Bestand und Bewegung

Fallstudie VeloCity, Schema velocity auf supabase.butscher.cloud. Quellen im Repository unter doku/datenmodell/.

Vier weitere Entscheidungen mit Begründung

Auch das Weglassen ist eine Entscheidung. Zwei der vier bestehen darin, etwas bewusst nicht zu tun.

1 Dauer ableiten folgt zwingend aus Start und Ende, wird nicht gepflegt

GENERATED-Spalte

2 Kein EAV generische Schlüssel-Wert-Tabellen verlieren alles Prüfbare

drei konkrete Tabellen

3 Modell zwischen Typ und Rad Ersatzteile hängen am Modell, nicht am Einzelrad

Brücke zur Warenwirtschaft

4 Positionen statt Betrag ein Betrag verrät nicht, wie er zustande kam

entgeltposition je Zeile

Fallstudie VeloCity, Schema velocity auf supabase.butscher.cloud. Quellen im Repository unter doku/datenmodell/.

3 · Normalisierung

Die Ausgangstabelle: alles in einer Relation

So würde jemand die Ausleihe aufschreiben, der noch nicht normalisiert hat. Wir zerlegen sie in drei Schritten und beobachten, welche Anomalie jeder Schritt beseitigt.

ausleihe_flach — der Ausgangszustand

ausleihe_id
kunde_email
kunde_name
kunde_strasse
kunde_plz
kunde_ort
rahmennummer
typ_bezeichnung
startgebuehr
preis_pro_minute
tageshoechstpreis
startzeit
endzeit
dauer_minuten
kosten
positionen

Die drei Befunde

- 1NF kunde_name trägt Vor- und Nachname
positionen trägt mehrere Zeilen
- 2NF kunde_email hängt nur an ausleihe_id,
nicht am ganzen Schlüssel
- 3NF rahmennummer -> typ_bezeichnung
-> preis_pro_minute

dauer_minuten und kosten sind
abgeleitet, nicht elementar

Fallstudie VeloCity, Schema velocity auf supabase.butscher.cloud. Quellen im Repository unter doku/datenmodell/.

Erste und zweite Normalform am Beispiel

Die 1NF verlangt atomare Attribute. Der lehrreichere Fall ist die wiederholende Gruppe: sie wird zu einer eigenen Relation, nicht zu mehreren Spalten.

1NF · atomar

kunde_name wird zu vorname und nachname
positionen wird eine eigene Relation

NICHT position1, position2, position3 —
das legt die Anzahl im Schema fest

2NF · keine partielle Abhängigkeit

Schlüssel (ausleihe_id, position_nr)
kunde_email hängt nur an ausleihe_id

Folge: Trennung in Kopf und Positionen
Das Kopf-Positionen-Muster ist keine
Konvention, sondern eine Folge der 2NF

Merksatz: Eine wiederholende Gruppe wird zu Zeilen, niemals zu Spalten.

Fallstudie VeloCity, Schema velocity auf supabase.butscher.cloud. Quellen im Repository unter [doku/datenmodell/](#).

Dritte Normalform — und warum sie hier nicht genügt

Die transitive Abhängigkeit vom Rad über den Typ zum Preis ist der klassische 3NF-Verstoß. Sie aufzulösen ist notwendig, reicht aber fachlich nicht aus.

Die transitive Kette

`ausleihe_id`

-> `rahmennummer`

-> `typ_bezeichnung`

-> `preis_pro_minute`

Änderungsanomalie: eine Preisänderung müsste in jeder Ausleihzeile nachgezogen werden.

Nach der Zerlegung — und danach

3NF: `fahrradtyp` und `nutzungspreis` werden eigene Relationen

ABER: liegt der Preis am Typ, ändert eine Anpassung weiterhin rückwirkend alle Altrechnungen

Erst der Gültigkeitszeitraum in `nutzungspreis` macht es richtig.

Normalisierung beseitigt Redundanz. Sie ersetzt keine fachliche Analyse.

Exkurs: aus der Postleitzahl folgt nicht der Ort

Der Lehrbuchklassiker lautet, plz bestimme ort, also gehöre ort in eine eigene Relation. In Deutschland stimmt das nicht — und der scheinbar saubere Zerlegungsschritt wäre fachlich falsch.

gilt

Eine PLZ hat genau einen Ort

×

Ländliche Sammel-PLZ umfassen mehrere Orte

Ein Ort hat genau eine PLZ

×

Großstädte haben Dutzende

plz -> ort ist funktional

×

Also keine transitive Abhängigkeit, kein Split

Eine funktionale Abhängigkeit ist eine Behauptung über die Wirklichkeit und muss geprüft werden — nicht aus Namensähnlichkeit geschlossen.

Fallstudie VeloCity, Schema velocity auf supabase.butscher.cloud. Quellen im Repository unter doku/datenmodell/.

4 · Logischer Entwurf

Die Abbildung folgt festen Regeln

Wer eine 1:N-Beziehung auf der Eins-Seite verankert, hat nicht anders entworfen, sondern falsch abgebildet. Genau deshalb ist dieser Schritt korrigierbar.

ERM-Konstrukt	Abbildung ins Relationenmodell	Beispiel im Modell
Entitätstyp	eigene Relation	kunde, station, ausleihe
Attribut	Spalte	vorname, kapazitaet
1:N-Beziehung	Fremdschlüssel auf der N-Seite	ausleihe.kunde_id
1:1-Beziehung	FK auf der optionalen Seite, dort zugleich PK	fahrrad_position.fahrrad_id
M:N-Beziehung	eigene Verknüpfungsrelation	in Phase 2: wartungsposition
Mehrwertiges Attribut	eigene Relation	fahrradtyp_merkmal

Fallstudie VeloCity, Schema velocity auf supabase.butscher.cloud. Quellen im Repository unter [doku/datenmodell/](#).

Jede Relation trägt zwei Schlüssel

Surrogatschlüssel als Primärschlüssel, fachlicher Schlüssel als Eindeutigkeitsbedingung. Beide haben eine Aufgabe, keiner ersetzt den anderen.

	stabil	eindeutig	sprechend	
Surrogatschlüssel kunde_id	✓	✓	✗	Primärschlüssel
Fachschlüssel kundenummer	✗	✓	✓	UNIQUE-Bedingung
Fachschlüssel als PK	✗	✓	✓	Änderung zieht durch alle Verweise

Eine Rahmennummer wird beim Rahmentausch neu vergeben. Ein Primärschlüssel, der sich ändert, zieht die Änderung durch jede verweisende Relation.

Fallstudie VeloCity, Schema velocity auf supabase.butscher.cloud. Quellen im Repository unter [doku/datenmodell/](#).

5 · Physischer Entwurf

Datentypen sind fachliche Entscheidungen

Jede Zeile dieser Tabelle hat einen Grund, der über Geschmack hinausgeht. Die beiden ersten sind die, an denen in der Praxis am meisten schiefeht.

Zweck	Typ	Warum genau dieser
Zeitpunkt	timestamptz	timestamp ohne Zone verliert den Bezug; Sommerzeit wird doppeldeutig
Geldbetrag	numeric(10,2)	exakte Dezimalarithmetik; float trifft 0,10 nicht genau
Zeitraum	daterange	ein Zeitraum ist ein Wert, kein Spaltenpaar — erst so prüfbar
Surrogatschlüssel	bigint IDENTITY	standardkonform; ALWAYS verhindert Setzen von außen
Koordinate	numeric(9,6)	sechs Nachkommastellen entsprechen etwa elf Zentimetern
Text	text	PostgreSQL speichert text und varchar(n) identisch

Fallstudie VeloCity, Schema velocity auf supabase.butscher.cloud. Quellen im Repository unter [doku/datenmodell/](#).

ENUM oder Referenztable — die Regel dahinter

Beide beschränken Werte. Die Wahl folgt nicht dem Geschmack, sondern drei Fragen: wächst die Menge, trägt sie Attribute, muss die Anwendung sie interpretieren?

	geschlossen	Attribute	wächst	
ENUM fahrrad_status	✓	×	×	verfuegbar, ausgeliehen, wartung, defekt
Referenztable entgeltart	×	✓	✓	trägt vorzeichen: belastet oder entlastet

Warum entgeltart kein ENUM ist

Die Preisfindung wertet das Attribut vorzeichen aus. Als ENUM müsste die Anwendung wissen, welche Art belastet und welche entlastet — das Wissen läge dann im Code statt in den Daten.

Fallstudie VeloCity, Schema velocity auf supabase.butscher.cloud. Quellen im Repository unter doku/datenmodell/.

Sieben von zehn Regeln erzwingt die Datenbank

Jede Regel, die nicht in einem Constraint, einem Index oder einer Funktion steht, wird früher oder später verletzt. Die drei übrigen brauchen Kontext, den ein Constraint nicht hat.

Regel	Umsetzung	Ort
GR1	CREATE UNIQUE INDEX ... WHERE status = 'aktiv'	partieller Index
GR3, GR4	EXCLUDE USING gist (... WITH =, ... WITH &&)	Constraint
GR6	GENERATED ALWAYS AS (ceil(...)) STORED	berechnete Spalte
GR7	CHECK (verbraucht <= kontingent)	Constraint
GR10	UNIQUE (kunde_id, periode_jahr, periode_monat)	Constraint
GR2, GR5	Prüfung in fn_ausleihe_starten / _beenden	Funktion
GR8, GR9	Prüfung in der api_-Schicht	Funktion

Fallstudie VeloCity, Schema velocity auf supabase.butscher.cloud. Quellen im Repository unter doku/datenmodell/.

EXCLUDE verhindert, was UNIQUE nicht kann

Ein UNIQUE kennt nur Gleichheit. Für „diese Zeiträume dürfen sich nicht überschneiden“ braucht es einen Operator, der Ueberlappung prüfen kann.

Der Constraint

```
constraint nutzungspreis_ueberschneidung_ex
  exclude using gist (
    typ_id          with =,
    gueltigkeit with &&
  )
```

Zu lesen als: es darf keine zwei Zeilen geben, bei denen typ_id gleich UND die Zeiträume überlappend sind.

Zwei Fallstricke

- 1 Voraussetzung btree_gist
Ohne die Erweiterung fehlt bigint die Operatorklasse für gist:
„data type bigint has no default operator class for access method gist“
- 2 Halboffene Grenzen '['']'
Sonst ist der Wechseltag doppelt belegt und der Constraint schlägt an.

Fallstudie VeloCity, Schema velocity auf supabase.butscher.cloud. Quellen im Repository unter doku/datenmodell/.

Warum das Mindestalter kein CHECK sein darf

Naheliegender wäre eine Bedingung mit `current_date`. PostgreSQL akzeptiert sie sogar. Trotzdem ist sie falsch — und der Grund zeigt, was ein CHECK eigentlich ist.

Verlockend und falsch

```
check (geburtsdatum <=
       current_date - interval '16 years')
```

`current_date` ist nicht IMMUTABLE.

Ein CHECK wird beim Schreiben geprüft
UND beim Wiedereinspielen eines Dumps.

Ein Kunde, der bei der Anmeldung 16 war,
bleibt es. Eine Bedingung, die sich auf
„heute“ bezieht, kann den Restore
abbrechen lassen.

Richtig

```
-- auf der Tabelle: nur immutable
check (geburtsdatum is null
       or geburtsdatum between
         date '1900-01-01'
         and date '2100-01-01')

-- die Altersregel in der Funktion:
if p_geburtsdatum >
    current_date - interval '16 years'
then
    return 'Mindestalter nicht erreicht';
end if;
```

Merksatz: Ein CHECK darf nur von der Zeile selbst abhängen — und von nichts, was sich mit der Zeit ändert.

Fallstudie VeloCity, Schema velocity auf supabase.butscher.cloud. Quellen im Repository unter doku/datenmodell/.

6 · Implementierung

Zwölf Aufbauschritte, jeder für sich lauffähig

Jede Datei ist idempotent: sie läuft zweimal hintereinander fehlerfrei. Das ist keine Spielerei, sondern die Voraussetzung dafür, dass man einen Aufbau gefahrlos wiederholen kann.

0001 Schema, Erweiterungen, Aufzählungstypen, Audit-Mechanik

0002 bis 0007 · die sechs Fachbereiche A bis F, 25 Tabellen

0008 Referenzdaten: Entgeltarten, Preise, Tarife, Inhalte

0009 Geschäftslogik: fn_-Fachlogik und api_-Zugriffsschicht

0010 Sichten · 0011 Zugriffsschutz · 0012 Data Dictionary

Fallstudie VeloCity, Schema velocity auf supabase.butscher.cloud. Quellen im Repository unter doku/datenmodell/.

Die Preisfindung als Folge sichtbarer Positionen

Das Zeitentgelt wird über ALLE Minuten gebildet und die Freiminuten als eigene Gutschrift abgezogen. So ist auf der Rechnung ablesbar, was der Tarifvorteil wert war.

Position	Menge	Einzelbetrag	Betrag	Beispiel 61 Minuten
STARTGEBUEHR	1	0,10	+ 0,10	Startgebühr City-Bike
ZEITENTGELT	61	0,10	+ 6,10	alle gefahrenen Minuten
FREIMINUTEN	30	0,10	– 3,00	Gutschrift aus dem Tarif
TARIFRABATT	1	20 %	– 0,64	auf die Zwischensumme
HOECHSTPREIS	1	—	– 0,00	erst über der Obergrenze

Reihenfolge: Rabatt VOR der Kappung. Umgekehrt würde der Rabatt den bereits gedeckelten Betrag ein zweites Mal senken.

Fallstudie VeloCity, Schema velocity auf supabase.butscher.cloud. Quellen im Repository unter doku/datenmodell/.

Dokumentation, die nicht veralten kann

Ein Data Dictionary, das von Hand gepflegt wird, ist nach dem zweiten Release falsch. Die Lösung ist, es aus dem Systemkatalog zu erzeugen — und die Vollständigkeit maschinell zu erzwingen.

Beschreibung an das Objekt

```
comment on column
  velocity.fahrrad_position.station_id is
  'NULL bedeutet: das Rad steht frei
  abgestellt, nicht an einer Station';

comment on column velocity.kunde.geburtsdatum
  is 'Grundlage der Altersgrenze. Geprüft
  in api_profil_aktualisieren, nicht
  per CHECK: current_date ist nicht
  immutable.';
```

Vollständigkeit erzwingen

```
-- Test schlägt fehl, solange eine
-- Fachspalte ohne Kommentar ist
select string_agg(c.relname||'.'||a.attname, ', ')
  from pg_class c
  join pg_attribute a on a.attrelid = c.oid
 where col_description(c.oid, a.attnum) is null
    and a.attname not in
      ('erstellt_am','geaendert_am');
```

316 Spalten, 264 mit Beschreibung,
52 technische Audit-Spalten ausgenommen.

7 · Zugriffsschutz

Der Schlüssel im Browser ist kein Geheimnis

Supabase liefert den anon-Key an jeden Besucher aus. Er steht im Quelltext der Seite. Jeder kann damit beliebige Anfragen stellen — mit curl, ohne die Website je zu öffnen.

Daraus folgt zwingend

Jede Zugriffsbeschränkung, die im JavaScript steht, ist wirkungslos.

Der Schutz muss in der Datenbank liegen: in Rechten, in Row Level Security und in Sichten, die nur zeigen, was öffentlich sein darf.

Das Antipattern

```
create policy "alles für alle"
  on kunde for all
  to anon, authenticated
  using (true) with check (true);
```

Folge: jeder mit dem öffentlichen Schlüssel liest UND ändert sämtliche Kundendaten.

Die Grundhaltung

```
-- RLS auf jeder Basistabelle
alter table ... enable row level security;
```

```
-- keine Policy für anon
-- Lesen nur über v_-Sichten
-- Schreiben nur über api_-Funktionen
```

```
default deny: was nicht ausdrücklich
erlaubt ist, ist verboten.
```

Fallstudie VeloCity, Schema velocity auf supabase.butscher.cloud. Quellen im Repository unter doku/datenmodell/.

Die Falle, die fast jeder übersieht

PostgreSQL vergibt das Ausführungsrecht auf jede neu angelegte Funktion automatisch an die Rolle PUBLIC. Ein Entzug gegen einzelne Rollen greift deshalb nicht.

Wirkungslos

```
revoke all on all functions  
in schema velocity  
from anon, authenticated;
```

anon und authenticated erben das Recht weiterhin über PUBLIC.

Folge: die interne Fachlogik
fn_ausleihe_beenden bleibt aufrufbar –
und die prüft selbst NICHT auf
auth.uid(), weil das die Schicht
darüber tut.

Wirksam

```
revoke all on all functions  
in schema velocity  
from public, anon, authenticated;
```

```
alter default privileges  
in schema velocity  
revoke execute on functions  
from public;
```

Die zweite Anweisung verhindert den
Rückfall bei künftigen Funktionen.

Aufgedeckt hat das ein Test, nicht Nachdenken. Rechte werden geprüft, nicht angenommen.

Fallstudie VeloCity, Schema velocity auf supabase.butscher.cloud. Quellen im Repository unter doku/datenmodell/.

Nachweis statt Behauptung — auf drei Wegen

Ein Sicherheitskonzept, das nur beschrieben ist, ist wertlos. Geprüft wird in der Datenbank, über die Schnittstelle und im Browser.

1 In der Datenbank RLS überall aktiv, anon ohne Tabellenrechte, Rollenwechsel-Probe

pgTAP, t0011

2 Ueber die Schnittstelle 13 gesperrte Ressourcen, 7 öffentliche Sichten

HTTP 401 gegen 200

3 Im Browser Aufrufe der abgemeldeten Seite in der Konsole

permission denied

Eine Falle im Prüfwerkzeug selbst

Der erste Entwurf meldete alle 13 gesperrten Ressourcen als bestanden — obwohl er nichts geprüft hatte: das Schema war gar nicht exponiert, also war alles unerreichbar, Sicheres wie Unsicheres.

Ein Test, der „abgesichert“ nicht von „gar nicht erreichbar“ unterscheidet, ist gefährlicher als kein Test.

Fallstudie VeloCity, Schema velocity auf supabase.butscher.cloud. Quellen im Repository unter doku/datenmodell/.

8 · Anwendung anbinden

Nur Sichten lesen, nur Funktionen schreiben

Die Anwendung kennt keine einzige Basistabelle. Das ist keine Konvention, sondern erzwungen: sie käme gar nicht an sie heran.

Lesen

```
from('v_station')
from('v_verfuegbares_fahrrad')
from('v_tarifkarte')
from('v_tarif')
from('v_faq')
from('v_nutzungsschritt')
from('v_kennzahl')

-- angemeldet:
from('v_meine_ausleihe')
from('v_mein_profil')
```

Schreiben

```
rpc('api_kunde_sicherstellen')
rpc('api_ausleihe_starten')
rpc('api_ausleihe_beenden')
rpc('api_profil_aktualisieren')

-- nicht erreichbar:
from('kunde')           permission denied
from('ausleihe')        permission denied
rpc('fn_ausleihe_beenden')
                        permission denied
```

9 · Zusammenfassung und Ausblick

Sechs Sätze, die diese Einheit tragen

Wenn Sie nichts anderes mitnehmen, dann diese sechs. Jeder einzelne ist an einer Stelle des Modells belegt.

1 Reihenfolge Anforderungen vor Modell, Modell vor Relationen, Relationen vor DDL

2 Kardinalität ist eine fachliche Aussage, keine technische Festlegung

3 Normalform ist notwendig, aber nicht hinreichend

4 Constraint was nicht erzwungen wird, wird verletzt

5 Schutz liegt in der Datenbank, nie im Browser

6 Nachweis Rechte werden geprüft, nicht angenommen

Fallstudie VeloCity, Schema velocity auf supabase.butscher.cloud. Quellen im Repository unter doku/datenmodell/.

Was die Warenwirtschaft ergänzt

Vier weitere Fachbereiche sind bereits entworfen. Sie hängen an denselben Entitäten — deshalb war es richtig, hersteller und fahrradmodell schon jetzt einzuziehen.

G Beschaffung

lieferant, artikelgruppe, artikel
bestellung, bestellposition
wareneingang

H Lager

lager, lagerbewegung
lagerbestand als Sicht

Bestand ist ein Aggregat von
Bewegungen, kein gepflegter Wert

I und J

schadensmeldung, wartungsauftrag
wartungsposition, fahrrad_ereignis
mitarbeiter, rolle
umsetzungsauftrag

Anschlussstellen: fahrrad zu Schadensmeldung und Wartung, station zum Umsetzungsauftrag, adresse zu Lieferant und Lager.

Fallstudie VeloCity, Schema velocity auf supabase.butscher.cloud. Quellen im Repository unter doku/datenmodell/.